

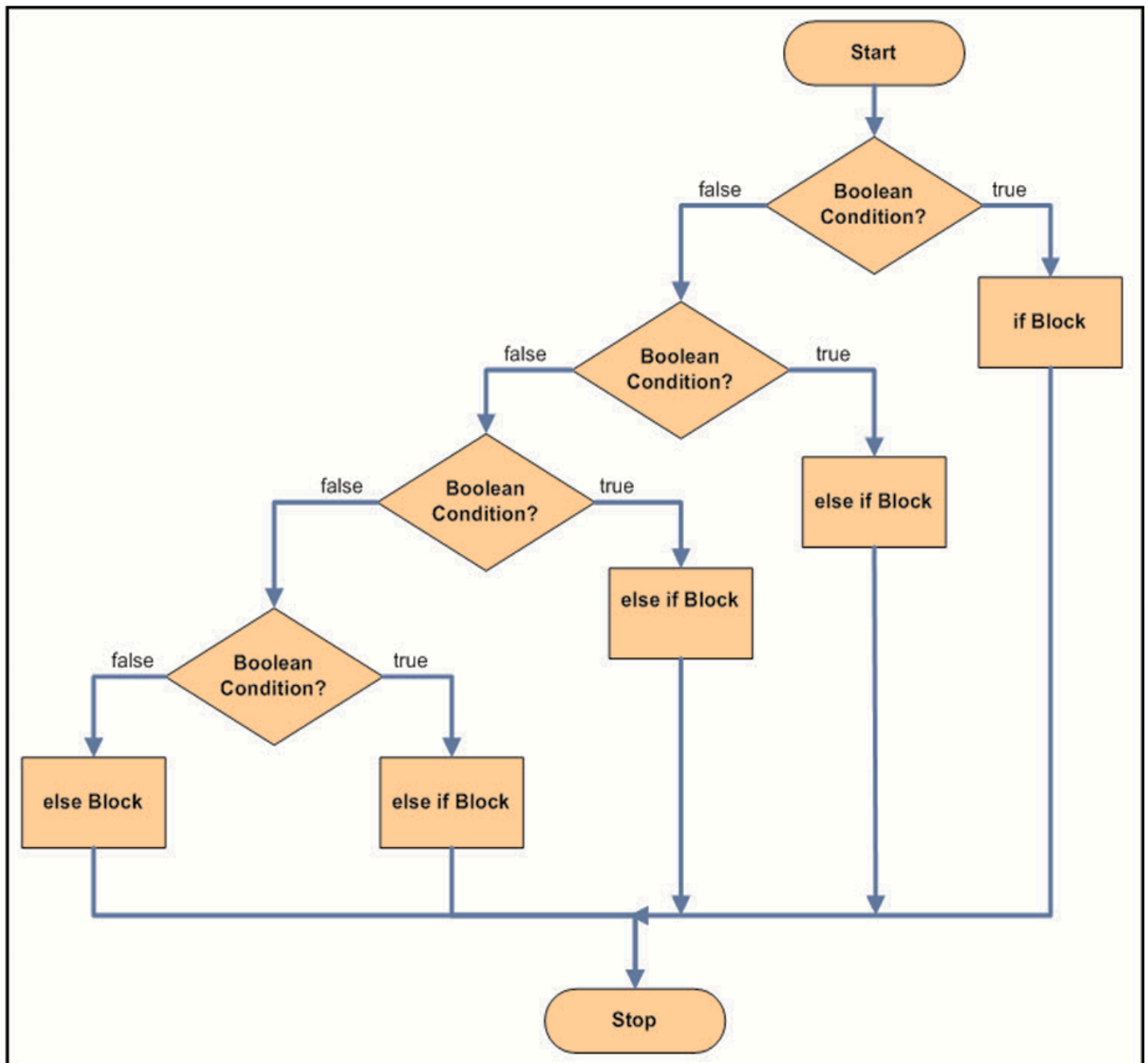
Session 2: Decision Making Constructs and Loops

In Java, control flow statements allow you to dictate the execution path of your program based on conditions, repetitions, or jumps. This session covers the fundamental constructs for decision-making, iteration, and branching. We divide the content into three modules: Conditional Statements, Loops, and Branching.

Control Flow

Module 1: Conditional Statements

Conditional statements enable decision-making in Java by evaluating boolean expressions and executing code blocks accordingly. They help programs respond dynamically to different inputs or states.



Types of Decision-Making Statements

Java provides several types of decision-making statements:

- **if statement:** Executes a block of code if a condition is true.
- **if-else statement:** Executes one block if true, another if false.
- **if-else-if ladder:** Checks multiple conditions sequentially.
- **Nested if:** An if statement inside another if.
- **switch-case statement:** Selects one of many code blocks based on a variable's value.
- **Ternary operator (? 🤪):** A shorthand for simple if-else (e.g., `int max = (a > b) ? a : b;`), though it's an operator, not a full statement.

These statements are essential for handling logic like user input validation, error checking, or branching based on data.

The if Statement and Its Variations

The `if` statement evaluates a boolean expression and executes the associated code block if it's true.

- **Basic if:**

```
if (condition) {  
    // Code to execute if condition is true  
}
```

Example:

```
int age = 18;  
if (age >= 18) {  
    System.out.println("You are an adult.");  
}
```

Best use: Simple checks where no alternative action is needed if false.

- **if-else:**

Adds an alternative block for when the condition is false.

```
if (condition) {  
    // True block  
} else {  
    // False block  
}
```

Example:

```
int score = 85;  
if (score >= 60) {  
    System.out.println("Passed");  
} else {  
    System.out.println("Failed");  
}
```

Best use: Binary decisions (e.g., pass/fail).

- **if-else-if ladder:**

Checks multiple conditions in sequence.

```

if (condition1) {
    // Block 1
} else if (condition2) {
    // Block 2
} else {
    // Default block
}

```

Example:

```

char grade = 'B';
if (grade == 'A') {
    System.out.println("Excellent");
} else if (grade == 'B') {
    System.out.println("Good");
} else {
    System.out.println("Needs improvement");
}

```

Best use: Mutually exclusive conditions, like grading systems.

- **Nested if:**

An if inside another if, for multi-level checks.

```

if (outerCondition) {
    if (innerCondition) {
        // Inner block
    }
}

```

Example:

```

boolean isCitizen = true;
int age = 20;
if (isCitizen) {
    if (age >= 18) {
        System.out.println("Eligible to vote");
    }
}

```

Best use: Hierarchical decisions (e.g., eligibility checks). Avoid deep nesting for readability; consider refactoring with methods.

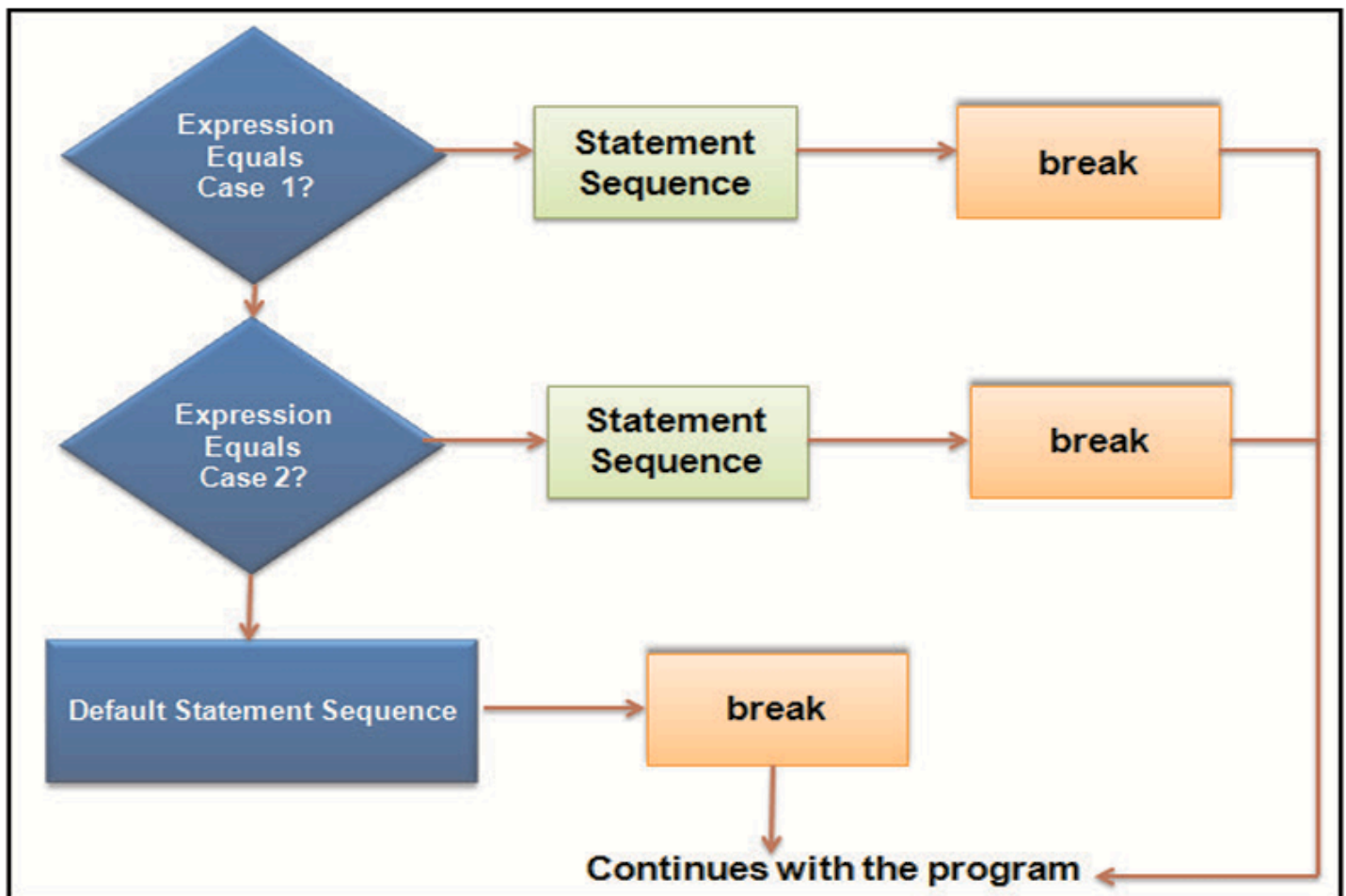
When to use if variations:

- Use basic `if` for optional actions.

- Add `else` for required alternatives.
- Use ladders for multiple options.
- Nest for complex, dependent conditions, but limit depth to 2-3 levels to maintain code clarity.

The switch-case Statement

The `switch` statement evaluates an expression (typically a variable) and matches it against multiple case values, executing the matching block. It supports `byte`, `short`, `int`, `char`, `String`, `enum`, and wrappers like `Integer`.



Syntax:

```

switch (expression) {
    case value1:
        // Code
        break;
    case value2:
        // Code
        break;
    default:
        // Default code
}

```

Example:

```
int day = 3;
switch (day) {
    case 1:
        System.out.println("Monday");
        break;
    case 2:
        System.out.println("Tuesday");
        break;
    case 3:
        System.out.println("Wednesday");
        break;
    default:
        System.out.println("Invalid day");
}
```

- **Fall-through:** Without `break` , execution continues to the next case (useful for shared logic).
- **Default:** Optional catch-all for unmatched values.
- **Enhanced switch (Java 12+):** Arrow syntax (`case value -> code;`) eliminates `break` and fall-through risks.

Example:

```
String result = switch (day) {
    case 1 -> "Monday";
    case 2 -> "Tuesday";
    default -> "Invalid";
};
```

Best use cases:

- When testing a single variable against many constants (e.g., menu options, enum handling).
- More readable than long if-else-if ladders for discrete values.

Comparison Between if-else and switch-case

Aspect	if-else/if-else-if	switch-case
Condition Type	Boolean expressions (e.g., ranges, complex logic like <code>age > 18 && isCitizen</code>)	Equality checks against constants (no ranges without tricks)
Data Types	Any boolean-evaluable	Limited (int, String, enum, etc.)

Aspect	if-else/if-else-if	switch-case
Readability	Good for 2-3 conditions; cluttered for many	Cleaner for many discrete values
Performance	Sequential checks ($O(n)$ in ladders)	Jump table ($O(1)$ for dense cases)
Flexibility	Handles ranges, inequalities, multiple variables	Strict matching; no complex logic
Fall-through	N/A	Possible (intentional or error-prone)
Default Handling	Explicit <code>else</code>	Optional <code>default</code>

When to use which:

- Use `if-else` for complex conditions, ranges (e.g., `if (score >= 90)`), or few options.
- Use `switch` for many equality checks on one variable (e.g., day of week). Switch is often faster and more maintainable for enums or constants.

Class Work

Write a Java program to determine a student's grade based on their score using both if-else-if ladder and switch-case. Use the following grading scale:

- 90-100: A
- 80-89: B
- 70-79: C
- 60-69: D
- Below 60: F

```
public class GradeChecker { public static void main(String[] args) { int score = 85; // Example score

    // Using if-else-if ladder
    if (score >= 90 && score <= 100) {
        System.out.println("Grade (if-else): A");
    } else if (score >= 80 && score <= 89) {
        System.out.println("Grade (if-else): B");
    } else if (score >= 70 && score <= 79) {
        System.out.println("Grade (if-else): C");
    } else if (score >= 60 && score <= 69) {
        System.out.println("Grade (if-else): D");
    } else {
        System.out.println("Grade (if-else): F");
    }

    // Using switch-case (convert score range to grade letter first)
    char grade;
    if (score >= 90) grade = 'A';
    else if (score >= 80) grade = 'B';
    else if (score >= 70) grade = 'C';
    else if (score >= 60) grade = 'D';
    else grade = 'F';

    switch (grade) {
        case 'A':
            System.out.println("Grade (switch): A");
            break;
        case 'B':
            System.out.println("Grade (switch): B");
            break;
        case 'C':
            System.out.println("Grade (switch): C");
            break;
        case 'D':
            System.out.println("Grade (switch): D");
            break;
        case 'F':
            System.out.println("Grade (switch): F");
            break;
        default:
            System.out.println("Invalid grade");
    }
}

}
```

Module 2: Loops

Loops allow repeated execution of code blocks, ideal for iterating over data, performing calculations, or waiting for conditions.

Types of Loops

Java offers:

- **while loop**: Condition-checked before execution.
- **do-while loop**: Condition-checked after execution (runs at least once).
- **for loop**: Compact for counted iterations.
- **Enhanced for loop (for-each)**: Simplified for collections/arrays.

The while Statement and Associated Rules

The `while` loop executes as long as a condition is true.

Syntax:

```
while (condition) {  
    // Code  
}
```

Example:

```
int i = 0;  
while (i < 5) {  
    System.out.println(i);  
    i++;  
}
```

Rules:

- Condition must be boolean.
- Update variables inside to avoid infinite loops.
- **Infinite while loop**: Use `while (true)` for intentional endless execution (e.g., servers, games). Exit with `break`.

Example:

```
while (true) {  
    // Code; break on condition  
}
```

Best use: When iteration count is unknown (e.g., reading input until EOF).

Purpose of the do-while Statement

Similar to while, but guarantees at least one execution since the condition is checked post-iteration.

Syntax:

```
do {  
    // Code  
} while (condition);
```

Example:

```
int i = 0;  
do {  
    System.out.println(i);  
    i++;  
} while (i < 5);
```

Purpose: When the loop body must run at least once (e.g., menu prompts where input is taken first).

The for Statement

The `for` loop is for known iteration counts, with initialization, condition, and increment in one line.

Syntax:

```
for (initialization; condition; update) {  
    // Code  
}
```

Example:

```
for (int i = 0; i < 5; i++) {  
    System.out.println(i);  
}
```

- **Comma operator:** Multiple initializations/updates (e.g., `for (int i=0, j=10; i<5; i++, j--)`).
- **Variations:** Omit parts (e.g., `for (; condition; )` for while-like).
- **Infinite for loop:** `for (;;) – equivalent to while (true) .`
- **Best use:** Counted loops (e.g., array traversal).

Enhanced for Loop (for-each)

Iterates over arrays or collections without indexes.

Syntax:

```
for (Type element : collection) {  
    // Code with element  
}
```

Example:

```
int[] arr = {1, 2, 3};  
for (int num : arr) {  
    System.out.println(num);  
}
```

Best use: Read-only iteration over lists/arrays; no index needed.

Nested Loops

Loops inside loops, for multi-dimensional processing.

Example:

```
for (int i = 0; i < 3; i++) {  
    for (int j = 0; j < 2; j++) {  
        System.out.println(i + "," + j);  
    }  
}
```

Best use: Matrices, patterns (e.g., star printing).

Comparison Between Different Types of Loops

Loop Type	Entry-Controlled?	Guaranteed Execution	Best For	When to Use
while	Yes	No	Unknown iterations	Condition-based, may skip loop
do-while	No	Yes (at least once)	Body must run first	User input loops

Loop Type	Entry-Controlled?	Guaranteed Execution	Best For	When to Use
for	Yes	No	Known count, compact syntax	Arrays, fixed iterations
for-each	Yes	No	Collections without modification	Simplified traversal

When to use which:

- **while/do-while**: Condition-driven (do-while if initial run needed).
- **for**: Counter-based.
- **for-each**: When you don't need indexes or modifications.
- Avoid nested loops if performance is critical ($O(n^2)$ complexity).

Class Work

Create a Java program that prints a 5x5 multiplication table using nested loops. Use both a traditional for loop and an enhanced for loop (where applicable) to demonstrate different approaches.

```
public class MultiplicationTable { public static void main(String[] args) { // Using nested for loops
System.out.println("Multiplication Table (Traditional For Loop):"); for (int i = 1; i <= 5; i++) { for (int j =
1; j <= 5; j++) { System.out.print(i * j + "\t"); } System.out.println(); }

    // Using enhanced for loop with an array (alternative approach)
    System.out.println("\nMultiplication Table (Enhanced For Loop with Array):")
    int[] numbers = {1, 2, 3, 4, 5};
    for (int i : numbers) {
        for (int j : numbers) {
            System.out.print(i * j + "\t");
        }
        System.out.println();
    }
}
```

Module 3: Branching

Branching statements alter loop or switch flow by jumping to specific points.

Purpose of Jump Statements

Jump statements (break, continue) transfer control out of loops/switches prematurely, preventing unnecessary iterations or handling exceptions in flow.

The break Statement

Exits the nearest enclosing loop or switch.

Example (in loop):

```
for (int i = 0; i < 10; i++) {  
    if (i == 5) {  
        break;  
    }  
    System.out.println(i); // Prints 0-4  
}
```

In switch: Prevents fall-through.

Best use: Early termination (e.g., search found).

The continue Statement

Skips the rest of the current iteration and jumps to the next.

Example:

```
for (int i = 0; i < 5; i++) {  
    if (i % 2 == 0) {  
        continue;  
    }  
    System.out.println(i); // Prints 1,3  
}
```

Best use: Skipping invalid data in loops.

Labeled Statements

Labels allow break/continue to target outer loops in nested scenarios.

Syntax:

```
outer: for (int i = 0; i < 3; i++) {  
    for (int j = 0; j < 3; j++) {  
        if (j == 1) {  
            break outer; // Exits both loops  
        }  
    }  
}
```

Use sparingly for clarity; best for complex nested logic (e.g., matrix searches).

Class Work

Write a Java program that uses nested loops to print numbers from 1 to 4 for both i and j. Use a labeled break to exit both loops when i and j both reach 3, and use continue to skip printing when the sum of i and j is even.

```
public class LabeledJumpExample { public static void main(String[] args) { outer: for (int i = 1; i <= 4;  
i++) { for (int j = 1; j <= 4; j++) { if (i == 3 && j == 3) { break outer; // Exit both loops } if ((i + j) % 2 ==  
0) { continue; // Skip even sums } System.out.println("i=" + i + ", j=" + j); } } }
```